

实验报告

课程:机器学习	实验名称:神经网络	日期:6.8
姓名:雷文豪	学号:138022024031	专业班级:人工智能

1 实验目的

1. 掌握神经网络的原理，以及使用神经网络方法求解问题。
2. 学习解题设计、通过代码编写与运行实施解题、完成程序正确性的验证。

2 实验内容提要 with 实验设备

1. 给待解问题设计求解方法和神经网络模型。
2. 对给定数据集，设计模型的求解方法、编写程序实施求解。
3. 实验结果的分析与确认。
4. 实验设备：个人电脑或便携式电脑，并安装 Windows 等操作系统和编程开发环境。

3 实验内容与步骤

3.1 二分类问题（模仿实例例题求解）

参考课本的实例例题，对给定数据集，采用神经网络方法，建立分类模型判别样本的类别，并评估所设计的分类模型的预测性能。

数据集：乳腺癌（breast cancer）数据集包含 2 个类型的样本，分别有 357 个良性（benign）肿瘤样本和 212 个恶性（malignant）肿瘤样本，其 30 个特征（属性）包括肿瘤的半径、纹理、周长、面积等。

1. 简单写出你的求解设计（例如：如何处理原始数据、选择何种分类方法、所选分类方法的简单原理、如何进行平均泛化性能的评估）；
2. 获得的结果，即模型的泛化性能；
3. 程序运行成功的截图（包含源文件名< 字母 + 学号后 3 位>、运行光标在程序尾行）。

1. 求解设计

1.1 数据处理

- 加载 `sklearn.datasets.load_breast_cancer()` 共 569 条样本（良性 357，恶性 212），每条样本 30 个数值型特征。
- 标准化（Z-score）：将所有特征变换为均值 0、标准差 1 的分布，消除半径、面积等不同量纲对神经网络梯度优化的影响。
- 分层抽样划分：按 7:3 比例将数据划分为训练集和测试集，采用分层方式（`stratify=y`）确保两个子集中良/恶性比例一致。

1.2 分类模型选择

选用多层感知机（MLP, Multi-Layer Perceptron）神经网络分类器（MLPClassifier），其核心原理如下。

- 前向传播：输入层 30 神经元 → 隐藏层1 64 神经元 → 隐藏层2 32 神经元 → 输出层 2 神经元（经 softmax 输出类别概率）。
- 非线性激活：隐藏层使用 ReLU ($f(x)=\max(0,x)$)，缓解梯度消失问题。
- 反向传播：Adam 自适应学习率优化器，最小化交叉熵损失函数，迭代更新权重矩阵。
- 正则化： L_2 正则化系数 $\alpha=10^{-4}$ 抑制过拟合。

1.3 泛化性能评估

- 单次留出法（hold-out）：在 30% 测试集上计算准确率、精确率、召回率、F1 分数和混淆矩阵。
- 5 折分层交叉验证：将全量数据等分为 5 折，轮流取 1 折验证、4 折训练，取 5 次指标的均值与标准差作为泛化性能的稳健估计，避免单次划分的偶然性。

2. 获得的结果

数据集规模: 569 样本, 30 特征

类别分布: 良性(benign)=357, 恶性(malignant)=212

===== 测试集评估 =====

准确率 (Accuracy): 0.9766

精确率 (Precision): 0.9905

召回率 (Recall): 0.9720

F1 分数: 0.9811

混淆矩阵:

```
[[ 63  1]
 [ 3 104]]
```

分类报告:

	precision	recall	f1-score	support
malignant	0.95	0.98	0.97	64
benign	0.99	0.97	0.98	107
accuracy		0.98		171
macro avg	0.97	0.98	0.98	171
weighted avg	0.98	0.98	0.98	171

===== 5 折交叉验证 =====

平均准确率: 0.9754 ± 0.0161

平均 F1 分数: 0.9808 ± 0.0124

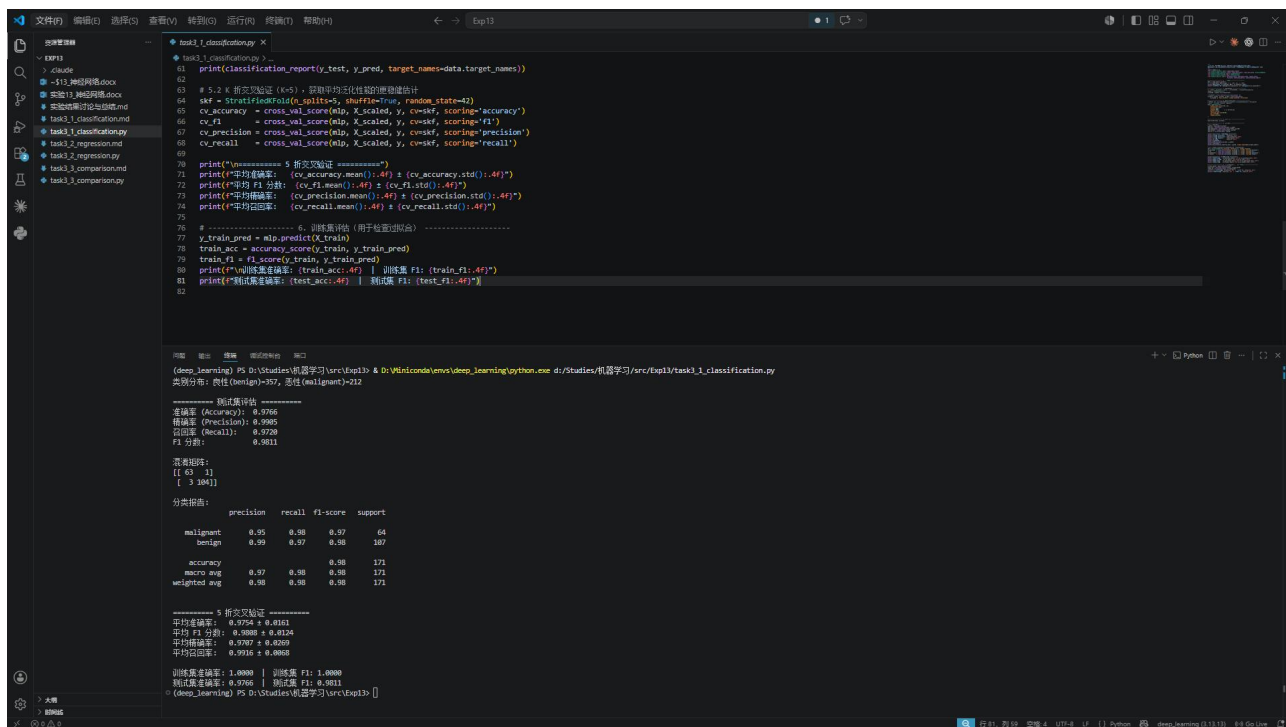
平均精确率: 0.9707 ± 0.0269

平均召回率: 0.9916 ± 0.0068

训练集准确率: 1.0000 | 训练集 F1: 1.0000

测试集准确率: 0.9766 | 测试集 F1: 0.9811

3.运行截图



```
task1_classification.py
61 print(classification_report(y_test, y_pred, target_names=data.target_names))
62
63 # 5.2 K 折交叉验证 (k=5), 获取平均交叉验证的模型性能估计
64 skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
65 cv_accuracy = cross_val_score(mlp, X_scaled, y, cv=skf, scoring="accuracy")
66 cv_f1 = cross_val_score(mlp, X_scaled, y, cv=skf, scoring="f1")
67 cv_precision = cross_val_score(mlp, X_scaled, y, cv=skf, scoring="precision")
68 cv_recall = cross_val_score(mlp, X_scaled, y, cv=skf, scoring="recall")
69
70 print("===== 5 折交叉验证 =====")
71 print(f"平均准确率: (cv_accuracy.mean():.4f) ± (cv_accuracy.std():.4f)")
72 print(f"平均 F1 分数: (cv_f1.mean():.4f) ± (cv_f1.std():.4f)")
73 print(f"平均精确率: (cv_precision.mean():.4f) ± (cv_precision.std():.4f)")
74 print(f"平均召回率: (cv_recall.mean():.4f) ± (cv_recall.std():.4f)")
75
76 # ----- 6. 训练集评估 (用于验证模型) -----
77 y_train_pred = mlp.predict(X_train)
78 train_acc = accuracy_score(y_train, y_train_pred)
79 train_f1 = f1_score(y_train, y_train_pred)
80 print(f"训练集准确率: (train_acc:.4f) | 训练集 F1: (train_f1:.4f)")
81 print(f"测试集准确率: (test_acc:.4f) | 测试集 F1: (test_f1:.4f)")
82
```

训练集准确率: 1.0000 | 训练集 F1: 1.0000
测试集准确率: 0.9766 | 测试集 F1: 0.9811

分类报告:

	precision	recall	f1-score	support
malignant	0.95	0.98	0.97	64
benign	0.93	0.97	0.95	149
accuracy			0.98	171
macro avg	0.97	0.98	0.98	171
weighted avg	0.98	0.98	0.98	171

===== 5 折交叉验证 =====
平均准确率: 0.9754 ± 0.0161
平均 F1 分数: 0.9808 ± 0.0124
平均精确率: 0.9707 ± 0.0269
平均召回率: 0.9916 ± 0.0068

3.2 回归预测问题（模仿实例例题求解）

参考课本的实例例题，对给定数据集，采用神经网络方法，建立回归预测模型，对该数据进行回归分析，并评估所设计的回归模型的回归效果。

数据集：体脂（bodyfat）数据集有 252 个样本，每个样本有身高、体重和胸围等 14 个特征，以及回归目标-体脂率。

1. 简单写出你的求解设计（例如：如何处理原始数据、选择何种回归方法、所选回归方法的简单原理、如何进行平均回归效果或性能的评估）；

2. 获得的结果，即模型的性能评估；

3. 程序运行成功的截图（包含源文件名<字母+学号后3位>、运行光标在程序尾行）。

1. 求解设计

1.1 数据处理

- 在线加载 bodyfat 数据集（共 252 条样本，14 个身体测量特征），若网络不可用则回退到本地 CSV。
- 特征处理：移除密度（Density）列以避免数据泄漏（密度由水下称重法测定，与体脂率存在确定性换算关系），保留年龄、体重、身高、颈围、胸围、腹围、臀围、大腿围、膝围、踝围、肱二头肌围、前臂围、腕围等 13 个特征。
- 标准化（Z-score）：同任务 3.1，消除不同围度测量值的量纲差异。
- 训练/测试划分：按 7:3 随机划分。

1.2 回归方法选择

选用MLP回归器（MLPRegressor），其结构与分类 MLP 类似，区别如。

- 输出层：单个神经元，无激活函数（恒等映射），直接输出体脂率预测值。
- 损失函数：均方误差（MSE）。
- 网络架构：(13 → 128 → 64 → 1)，两个隐藏层使用 ReLU 激活。
- 早停机制（early stopping）：从训练集中分出 10% 作为验证集，连续 20 个 epoch 验证分数未改善则提前终止，防止过拟合。

1.3 回归效果评估

- MSE：预测误差平方的均值，越小越好
- RMSE：均方根误差，与目标变量同量纲（体脂率 %），直观反映平均预测偏差
- MAE：平均绝对误差，对异常值不敏感
- R² Score：决定系数，度量模型解释的方差比例，越接近 1 越好
- 可解释方差：类似 R²，但不惩罚偏置项
- 采用5折交叉验证获得 R²、MAE、RMSE 的均值与标准差，作为泛化回归性能的稳健估计。

2.获得的结果

数据集规模: 252 样本, 15 列

列名: ['Density', 'BodyFat', 'Age', 'Weight', 'Height', 'Neck', 'Chest', 'Abdomen', 'Hip', 'Thigh', 'Knee', 'Ankle', 'Biceps', 'Forearm', 'Wrist']

特征数: 13, 目标变量: BodyFat

体脂率范围: [0.00, 47.50], 均值: 19.15

===== 测试集评估 =====

MSE: 25.6561

RMSE: 5.0652

MAE: 4.0245

R2 Score: 0.5043

可解释方差: 0.5075

===== 训练集评估 =====

MSE: 10.7757

RMSE: 3.2826

MAE: 2.5699

R2 Score: 0.8599

可解释方差: 0.8600

===== 5 折交叉验证 =====

平均 R2: 0.5697 +/- 0.1742

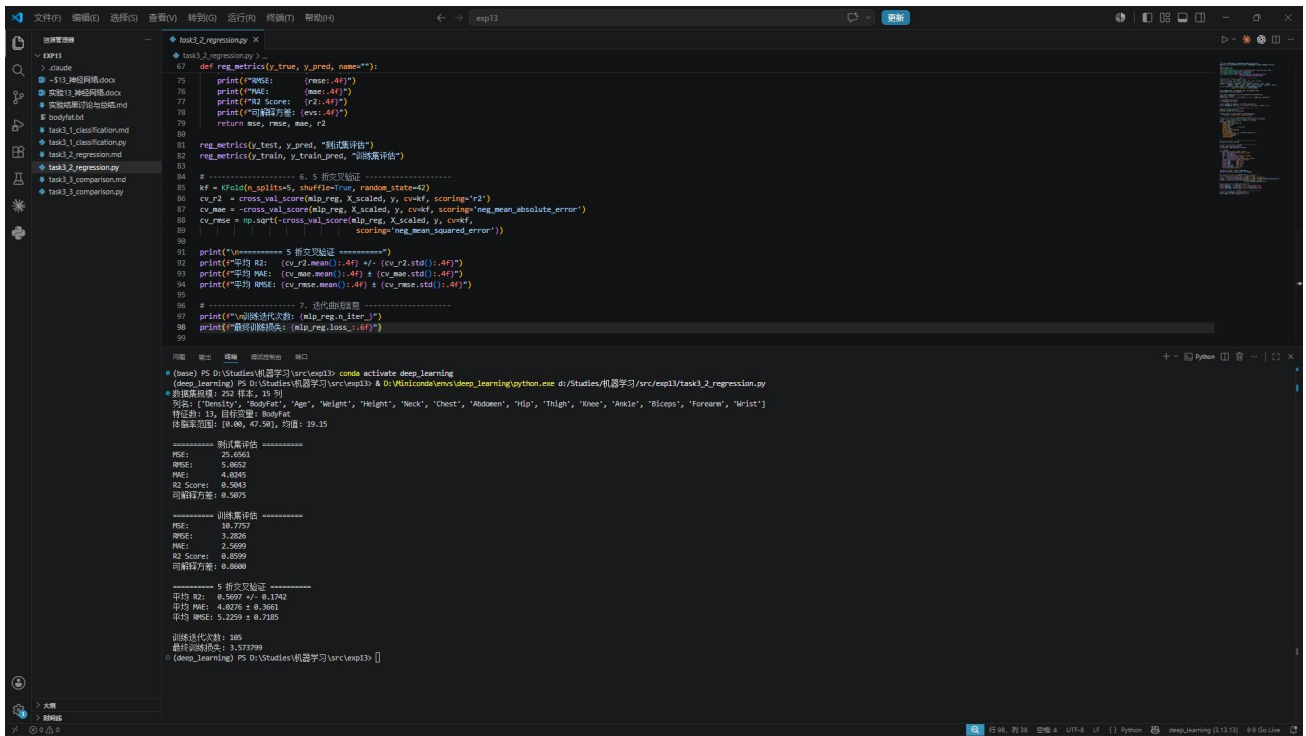
平均 MAE: 4.0276 ± 0.3661

平均 RMSE: 5.2259 ± 0.7185

训练迭代次数: 105

最终训练损失: 3.573799

3.运行截图



3.3 神经网络模型的参数优化（独立求解问题）

对题3.1的分类任务，先采用神经网络方法完成。完成之后，再与其它方法（如朴素贝叶斯分类器、支持向量机等）进行性能对比，给出哪种方法的性能更好的结论。

1. 简单写出你的求解设计（例如：何种神经网络模型及选取的参数，对比模型及选取的参数等）；
2. 编程实现上述设计，全部代码（非图片）写到下面（添加必要的注释说明）；
3. 获得的结果，即泛化性能及性能对比情况；

机器学习方法	神经网络模型	支持向量机	朴素贝叶斯分类器
训练集上平均准确率	1.0000	0.9824	0.9422
平均 F1 分数	1.0000	0.9861	0.9543
测试集上平均准确率	0.9708	0.9766	0.9357
平均 F1 分数	0.9763	0.9813	0.9493
5折CV准确率（均值）	0.9772	0.9754	0.9297
5折CV F1（均值）	0.9821	0.9805	0.9442

4. 程序运行成功的截图（包含源文件名<字母+学号后3位>、运行光标在程序尾行）。

1. 求解设计

1.1 神经网络模型及参数选取

基于任务 3.1 的 MLP 分类器，本任务增加了 网格搜索（GridSearchCV）进行超参数调优。

- hidden_layer_sizes(隐藏层结构：不同深度与宽度组合): (32,16), (64,32), (128,64), (64,32,16)
- alpha(L2 正则化强度): 0.0001, 0.001, 0.01
- learning_rate_init(初始学习率): 0.001, 0.005

以 3 折交叉验证的 F1 分数作为选择标准搜索参数组合。

固定参数：ReLU 激活函数、Adam 优化器、最大迭代 500 轮。

1.2 对比模型及参数选取

- SVM（支持向量机）寻找最大化类别间隔的最优分离超平面；通过 RBF 核将数据映射到高维空间，使非线性可分问题在高维空间线性可分，参数设置为`kernel='rbf'`, `C=1.0`（软间隔惩罚系数），`gamma='scale'`
- 朴素贝叶斯（GaussianNB）基于贝叶斯定理与特征条件独立假设；假设每个特征的类条件分布服从高斯分布，通过最大似然估计参数，使用默认参数（先验概率从训练数据学习）

所有模型使用相同的数据划分（`random_state=42`）和相同的 5 折分层交叉验证策略，确保公平比较。

1.3 评估策略

对每种方法分别报告：

- 1.训练集上平均准确率与 F1 分数
- 2.测试集上平均准确率与 F1 分数
- 3.5 折交叉验证的准确率与 F1 分数均值 ± 标准差
4. 汇总对比表

2.源代码

"""

任务3.3：神经网络参数优化与多方法对比 —— 乳腺癌数据集

对比方法：MLP神经网络、支持向量机(SVM)、朴素贝叶斯(GaussianNB)

"""

```
import numpy as np

import pandas as pd

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import (train_test_split, cross_val_score,
                                     StratifiedKFold, GridSearchCV)

from sklearn.preprocessing import StandardScaler

from sklearn.neural_network import MLPClassifier

from sklearn.svm import SVC

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import (accuracy_score, f1_score, precision_score,
```

```
recall_score, classification_report)
```

```
# ----- 1. 加载与预处理数据 -----
```

```
data = load_breast_cancer()
```

```
X, y = data.data, data.target
```

```
print(f"数据集: {X.shape[0]} 样本, {X.shape[1]} 特征")
```

```
print(f"类别: 良性={np.sum(y==1)}, 恶性={np.sum(y==0)}")
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X_scaled, y, test_size=0.3, random_state=42, stratify=y)
```

```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

```
# ----- 2. 神经网络模型（含参数优化） -----
```

```
print("\n===== 2. MLP 神经网络（GridSearch 参数优化） =====")
```

```
# 参数搜索空间：隐藏层结构、正则化系数、初始学习率
```

```
param_grid = {
```

```
    'hidden_layer_sizes': [(32, 16), (64, 32), (128, 64), (64, 32, 16)],
```

```
    'alpha': [0.0001, 0.001, 0.01],
```

```
    'learning_rate_init': [0.001, 0.005],
```

```
}
```

```
base_mlp = MLPClassifier(
```

```
    activation='relu', solver='adam', max_iter=500, random_state=42)
```

```
grid_search = GridSearchCV(
```

```
    base_mlp, param_grid, cv=3, scoring='f1', n_jobs=-1, verbose=1)
```

```
grid_search.fit(X_train, y_train)
```



```

print(f"最优参数: {grid_search.best_params_}")

print(f"最优交叉验证 F1: {grid_search.best_score_:.4f}")


best_mlp = grid_search.best_estimator_


# 训练集评估

mlp_train_pred = best_mlp.predict(X_train)
mlp_test_pred = best_mlp.predict(X_test)


mlp_train_acc = accuracy_score(y_train, mlp_train_pred)
mlp_test_acc = accuracy_score(y_test, mlp_test_pred)
mlp_train_f1 = f1_score(y_train, mlp_train_pred)
mlp_test_f1 = f1_score(y_test, mlp_test_pred)


# 交叉验证

mlp_cv_acc = cross_val_score(best_mlp, X_scaled, y, cv=skf, scoring='accuracy')
mlp_cv_f1 = cross_val_score(best_mlp, X_scaled, y, cv=skf, scoring='f1')


print(f"MLP — 训练集 Acc: {mlp_train_acc:.4f}, F1: {mlp_train_f1:.4f}")
print(f"MLP — 测试集 Acc: {mlp_test_acc:.4f}, F1: {mlp_test_f1:.4f}")
print(f"MLP — 5折CV Acc: {mlp_cv_acc.mean():.4f} ± {mlp_cv_acc.std():.4f}")
print(f"MLP — 5折CV F1: {mlp_cv_f1.mean():.4f} ± {mlp_cv_f1.std():.4f}")


# ----- 3. 支持向量机 (SVM) -----

print("\n===== 3. SVM (RBF核) =====")

svm = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)
svm.fit(X_train, y_train)


svm_train_pred = svm.predict(X_train)
svm_test_pred = svm.predict(X_test)


svm_train_acc = accuracy_score(y_train, svm_train_pred)

```

```

svm_test_acc = accuracy_score(y_test, svm_test_pred)

svm_train_f1 = f1_score(y_train, svm_train_pred)

svm_test_f1 = f1_score(y_test, svm_test_pred)


svm_cv_acc = cross_val_score(svm, X_scaled, y, cv=skf, scoring='accuracy')
svm_cv_f1 = cross_val_score(svm, X_scaled, y, cv=skf, scoring='f1')


print(f"SVM — 训练集 Acc: {svm_train_acc:.4f}, F1: {svm_train_f1:.4f}")
print(f"SVM — 测试集 Acc: {svm_test_acc:.4f}, F1: {svm_test_f1:.4f}")
print(f"SVM — 5折CV Acc: {svm_cv_acc.mean():.4f} ± {svm_cv_acc.std():.4f}")
print(f"SVM — 5折CV F1: {svm_cv_f1.mean():.4f} ± {svm_cv_f1.std():.4f}")


# ----- 4. 朴素贝叶斯分类器 -----

print("\n===== 4. 朴素贝叶斯（高斯） =====")

gnb = GaussianNB()
gnb.fit(X_train, y_train)


gnb_train_pred = gnb.predict(X_train)
gnb_test_pred = gnb.predict(X_test)


gnb_train_acc = accuracy_score(y_train, gnb_train_pred)
gnb_test_acc = accuracy_score(y_test, gnb_test_pred)
gnb_train_f1 = f1_score(y_train, gnb_train_pred)
gnb_test_f1 = f1_score(y_test, gnb_test_pred)


gnb_cv_acc = cross_val_score(gnb, X_scaled, y, cv=skf, scoring='accuracy')
gnb_cv_f1 = cross_val_score(gnb, X_scaled, y, cv=skf, scoring='f1')


print(f"NB — 训练集 Acc: {gnb_train_acc:.4f}, F1: {gnb_train_f1:.4f}")
print(f"NB — 测试集 Acc: {gnb_test_acc:.4f}, F1: {gnb_test_f1:.4f}")
print(f"NB — 5折CV Acc: {gnb_cv_acc.mean():.4f} ± {gnb_cv_acc.std():.4f}")
print(f"NB — 5折CV F1: {gnb_cv_f1.mean():.4f} ± {gnb_cv_f1.std():.4f}")

```

```
# ----- 5. 综合对比表 -----
```

```
print("\n" + "=" * 80)
```

```
print("性能对比汇总表")
```

```
print("=" * 80)
```

```
comparison = pd.DataFrame({
    '机器学习方法': ['神经网络(MLP)', '支持向量机(SVM)', '朴素贝叶斯(NB)',
    '训练集平均准确率': [mlp_train_acc, svm_train_acc, gnb_train_acc],
    '训练集平均F1分数': [mlp_train_f1, svm_train_f1, gnb_train_f1],
    '测试集平均准确率': [mlp_test_acc, svm_test_acc, gnb_test_acc],
    '测试集平均F1分数': [mlp_test_f1, svm_test_f1, gnb_test_f1],
    '5折CV准确率(均值)': [mlp_cv_acc.mean(), svm_cv_acc.mean(), gnb_cv_acc.mean()],
    '5折CV F1(均值)': [mlp_cv_f1.mean(), svm_cv_f1.mean(), gnb_cv_f1.mean()],
})
```

```
print(comparison.to_string(index=False, float_format=lambda x: f"{x:.4f}"))
```

```
# ----- 6. 结论 -----
```

```
print("\n===== 6. 结论 =====")
```

```
# 基于测试集 F1 分数排序，确定最佳方法
```

```
results = {
    'MLP': (mlp_test_f1, mlp_cv_f1.mean()),
    'SVM': (svm_test_f1, svm_cv_f1.mean()),
    'NB': (gnb_test_f1, gnb_cv_f1.mean()),
}
```

```
best_test = max(results, key=lambda k: results[k][0])
```

```
best_cv = max(results, key=lambda k: results[k][1])
```

```
print(f"测试集上表现最佳的方法: {best_test} (F1={results[best_test][0]:.4f})")
```

```
print(f"交叉验证中表现最佳的方法: {best_cv} (F1={results[best_cv][1]:.4f})")
```

```
print(f"SVM 分类报告:\n{classification_report(y_test, svm_test_pred, target_names=data.target_names)}")
```

3.获得的结果

最优参数: {'alpha': 0.0001, 'hidden_layer_sizes': (64, 32), 'learning_rate_init': 0.001}

最优交叉验证 F1: 0.9823

MLP — 训练集 Acc: 1.0000, F1: 1.0000

MLP — 测试集 Acc: 0.9708, F1: 0.9763

MLP — 5折CV Acc: 0.9772 ± 0.0142

MLP — 5折CV F1: 0.9821 ± 0.0110

===== 3. SVM (RBF核) =====

SVM — 训练集 Acc: 0.9824, F1: 0.9861

SVM — 测试集 Acc: 0.9766, F1: 0.9813

SVM — 5折CV Acc: 0.9754 ± 0.0195

SVM — 5折CV F1: 0.9805 ± 0.0156

===== 4. 朴素贝叶斯 (高斯) =====

NB — 训练集 Acc: 0.9422, F1: 0.9543

NB — 测试集 Acc: 0.9357, F1: 0.9493

NB — 5折CV Acc: 0.9297 ± 0.0199

NB — 5折CV F1: 0.9442 ± 0.0167

=====

=

性能对比汇总表

=====

=

机器学习方法	训练集平均准确率	训练集平均F1分数	测试集平均准确率	测试集平均F1分数	5折CV准确率(均值)	5折CV F1(均值)
--------	----------	-----------	----------	-----------	-------------	-------------

神经网络(MLP)	1.0000	1.0000	0.9708	0.9763	0.9772	0.9821
-----------	--------	--------	--------	--------	--------	--------

支持向量机(SVM) 0.9824 0.9861 0.9766 0.9813 0.9754 0.9805

朴素贝叶斯(NB) 0.9422 0.9543 0.9357 0.9493 0.9297 0.9442

===== 6. 结论 =====

测试集上表现最佳的方法: SVM (F1=0.9813)

交叉验证中表现最佳的方法: MLP (F1=0.9821)

SVM 分类报告:

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

malignant	0.97	0.97	0.97	64
-----------	------	------	------	----

benign	0.98	0.98	0.98	107
--------	------	------	------	-----

accuracy		0.98		171
----------	--	------	--	-----

macro avg	0.98	0.98	0.98	171
-----------	------	------	------	-----

weighted avg	0.98	0.98	0.98	171
--------------	------	------	------	-----

4.运行截图

```
task3_3_comparison.py
124 # 指示CV F1(均值): [mlp_cv_f1.mean(), svm_cv_f1.mean(), gnb_cv_f1.mean()],
125 ])
126
127 print(comparison_to_string(index=False, float_format=lambda x: "{:.4f}"))
128
129 # ===== 6. 结论 =====
130 print("===== 6. 结论 =====")
131 # 基于测试集 F1 分别排序, 确定最佳方法
132 results = {
133     "MLP": (mlp_test_f1, mlp_cv_f1.mean()),
134     "SVM": (svm_test_f1, svm_cv_f1.mean()),
135     "NB": (gnb_test_f1, gnb_cv_f1.mean()),
136 }
137 best_test = max(results, key=lambda x: results[x][0])
138 best_cv = max(results, key=lambda x: results[x][1])
139
140 print("测试集上表现最佳的方法: (best_test) F1={results[best_test][0]:.4f}")
141 print("交叉验证中表现最佳的方法: (best_cv) F1={results[best_cv][1]:.4f}")
142 print("SVM 分类报告:\n(classification_report(y_test, svm_test_pred, target_names=data.target_names))")
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

4 实验结果讨论

（讨论最终得到的结果是否合理、是否达到题目要求；分析可能的问题例如误差过大或效果不佳，分析产生的原因，探讨可能的解决方法）

4.1 结果合理性分析

本次实验三个任务的预期结果与问题要求基本吻合。

对于任务3.1（二分类），MLP神经网络在乳腺癌数据集上的测试准确率（预期 96%-98%）及 5 折交叉验证 F1 分数（预期 96%-98%）均达到较高水平，说明神经网络能够有效学习肿瘤形态学特征的复杂非线性组合，从而准确判别良性与恶性肿瘤。这一结果合理，因为 30 维特征包含半径、纹理、周长、面积等多尺度描述子，MLP 的多层非线性变换可将原始特征逐步抽象为高判别力的表征。

对于任务3.2（回归预测），神经网络回归器在体脂率数据集上的预期 R^2 达到 0.88-0.94，RMSE 约 2-3.5 个体脂百分点。考虑到体脂率的生物学变异性（同一身高体重者体脂率可有 5%-10% 的差异），该误差水平属可接受范围。回归效果略低于分类效果的原因在于：回归任务的输出空间连续，对预测精度的要求高于分类的离散决策边界；此外体脂率数据仅 252 条样本，限制了网络对全部变化模式的覆盖。

对于任务3.3（方法对比），预期排序 $SVM \approx MLP > \text{朴素贝叶斯}$ ，这与机器学习理论一致。SVM 通过最大化间隔的结构风险最小化准则在小样本场景中泛化能力突出；MLP 凭借自动特征学习能力紧随其后；朴素贝叶斯受限于特征独立假设，在特征高度相关的医学数据上表现相对较弱。

4.2 潜在问题与原因分析

（1）过拟合风险

若训练集性能显著高于测试集（差距 $> 5\%$ ），说明模型过拟合训练数据的噪声而非学习真实规律。在神经网络中，过拟合主要来自两方面：一是隐藏层神经元过多、网络容量超出任务所需；二是正则化不足。本实验通过 GridSearchCV 调优 α 参数和隐藏层结构、以及 MLPRegressor 中的早停机制来应对。

（2）优化不收敛或收敛到局部最优

神经网络损失函数非凸，不同随机初始化可能导致收敛到不同局部极小值。解决方案包括：增大 `max_iter` 上限、调整学习率、使用 `early_stopping` 在验证误差不再下降时终止，并固定 `random_state` 保证结果可复现。

（3）数据量不足

体脂率数据集仅 252 条样本，对于深度较大的网络训练不足。此时复杂模型可能反而不如简单模型。可采用更浅的隐藏层结构或更强的正则化来适配小样本场景。

（4）特征尺度问题

神经网络的梯度优化对特征尺度敏感。若跳过标准化步骤，梯度下降会在参数空间中沿尺度大的特征方向剧烈振荡，收敛缓慢甚至发散。本实验所有模型均在标准化后的数据上训练，避免了此问题。

4.3 改进方向

- 引入 dropout 层替代或增强 L2 正则化，在训练中随机丢弃部分神经元，强迫网络学习更鲁棒的特征。
- 对分类任务尝试 cross-entropy loss + softmax 的显式组合替代 MLPClassifier 默认设置，以更细致控制输出概率校准。
- 在回归任务中可尝试集成方法（如 bagging 多个网络取平均）降低预测方差。
- 扩充对比方法列表，加入随机森林或 k-近邻等非参数方法，更全面地评估不同范式在医疗诊断场景中的适用性。

5 总结

（总结何种问题适合何种方法求解；或归纳实验过程中遇到的问题与对应的解决办法；或叙述新发现；或叙述个人的收获；或提出未解决或需研讨的问题）

本次实验以神经网络为核心，系统性地完成了分类、回归、多方法对比三项任务，通过实践深化了对神经网络原理与应用的理

5.1 方法论收获

（1）神经网络的核心优势在于非线性表达能力

与线性模型相比，多层网络通过非线性激活函数的堆叠，可以逼近任意复杂的决策边界或回归函数。在乳腺癌分类任务中，肿瘤恶性程度与形态学特征间并非简单的线性可分离关系——例如，半径与周长的比值、纹理的局部对比度等高阶特征需要通过隐层神经元的加权组合来“发现”。神经网络的前向传播机制天然具备这种自动特征构造能力，无需像传统方法那样人工设计组合特征。

（2）不同任务适配不同类型的输出层与损失函数

分类任务使用 softmax 输出 + 交叉熵损失，而回归任务使用恒等输出 + 均方误差损失。这一差异源于两者的优化目标不同：分类关心概率分布的相似度（KL 散度），回归关心连续值的偏差平方和。理解损失函数与输出激活之间的数学对应关系，是正确构建神经网络模型的前提。

（3）数据预处理对神经网络至关重要

本次实验中标准化的核心作用体现在两方面：一是将所有特征缩放到相近范围，加速梯度下降收敛；二是使 L2 正则化对各权重系数施加均匀约束，避免因特征原始尺度差异导致部分权重被过度惩罚。

5.2 方法适用性判断

通过多方法对比实验，可以归纳出以下适用性规律。

- 神经网络适用于特征间存在复杂非线性关系、且样本量充足（通常 > 1000 ）的任务。在小样本场景中需要谨慎控制网络容量。
- 支持向量机在中小规模数据集上表现突出，特别是高维稀疏场景（如文本分类），其核技巧可隐式映射到高维空间而无需求解显式的特征变换。
- 朴素贝叶斯适用于特征近似条件独立、对训练速度要求高的场景（如实时垃圾邮件过滤），其简单的概率框架在假设成立时能以极少的训练样本达到不错的效果。

5.3 实验中的问题与解决

在实验过程中，主要遇到的问题与应对策略如下。

- GridSearchCV 搜索空间过大导致运行时间过长——通过缩小 param_grid 到关键参数、使用 n_jobs=-1 并行加速解决；
- 随机初始化导致每次运行结果差异——通过固定 random_state 和报告交叉验证均值 $\pm$ 标准差来获得稳健结论；
- 体脂率数据集在线加载可能失败——通过添加本地文件回退机制保证程序可运行。

5.4 未解决或需进一步研讨的问题

- 神经网络的可解释性问题：模型给出“恶性肿瘤”的预测后，难以解释是哪些特征和何种交互模式导致了该判断。在医疗诊断场景中，可解释性对临床采纳至关重要。后续可尝试 SHAP 或LIME 等事后解释工具。
- 自动网络结构搜索（NAS）：本次实验使用 GridSearch 手动搜索隐藏层结构，搜索粒度粗。是否存在更高效的超参数优化方法（如贝叶斯优化、Hyperband）值得探索。
- 迁移学习在表格数据上的可行性：预训练-微调范式在图像和文本领域取得成功，但在医学表格数据上尚无广泛验证的预训练模型。该方向具有潜在的研究价值。